

1. What are Errors?

प्रोग्रामिंग की दुनिया में, **Error** एक ऐसी स्थिति है जब कोई प्रोग्राम उम्मीद के मुताबिक काम नहीं करता है। यह कोडिंग के दौरान की गई गलती, गलत लॉजिक या सिस्टम संसाधनों की कमी के कारण हो सकता है।

सामान्यतः एरर्स को तीन मुख्य श्रेणियों में बांटा जाता है:

- 1. **Compile-Time (Syntax) Errors:** जब कोड लिखने के नियमों (व्याकरण) में गलती हो। कंपाइलर इसे पहले ही पकड़ लेता है।
 - *Example:* `int x = 10 (बिना ; के)।`
- 2. **Runtime Errors:** जब प्रोग्राम कंपाइल तो हो जाता है, लेकिन चलने के दौरान क्रैश हो जाता है।
 - *Example:* किसी संख्या को 0 से भाग देना।
- 3. **Logical Errors:** प्रोग्राम चलता है, लेकिन परिणाम गलत आता है।
 - *Example:* `2 + 2` का उत्तर 5 आना (गलत फॉर्मूला)।

Java Errors

- 4. जावा में **"Error"** शब्द का एक विशेष तकनीकी अर्थ है। जावा की `Throwable` क्लास के दो मुख्य हिस्से होते हैं: **Exception** और **Error**।
- 5. **Java Error (`java.lang.Error`) की परिभाषा:** यह उन गंभीर और लाइलाज (Irrecoverable) समस्याओं को दर्शाता है जो प्रोग्राम के नियंत्रण से बाहर होती हैं। ये समस्याएँ आमतौर पर जावा वर्चुअल मशीन (JVM) या हार्डवेयर के स्तर पर होती हैं।
- 6. **मुख्य विशेषता:** आप इन्हें `try-catch` ब्लॉक से ठीक नहीं कर सकते। यदि जावा एरर आता है, तो प्रोग्राम का बंद होना तय है।

Types of Java Errors (जावा एरर के प्रकार)

- 7. जावा में मुख्य रूप से निम्नलिखित प्रकार के एरर पाए जाते हैं:

Error Type	Description
VirtualMachineError	जब JVM खराब हो जाए या उसे काम जारी रखने के लिए संसाधन न मिलें।
OutOfMemoryError	जब प्रोग्राम इतने ऑब्जेक्ट बना दे कि RAM (Heap Space) भर जाए।
StackOverflowError	जब कोई मेथड खुद को इतनी बार कॉल करे (Infinite Recursion) कि मेमोरी स्टैक भर जाए।
NoClassDefFoundError	जब कंपाइलर की गई क्लास फाइल रनटाइम के दौरान गायब हो जाए।

Java Exception (The Exception Class)

जावा में, एक **Exception** वह घटना (Event) है जो प्रोग्राम के चलने (**Execution**) के दौरान होती है और निर्देशों के सामान्य प्रवाह (Normal Flow) को बाधित करती है। तकनीकी रूप से, Exception एक **Class** है जो `java.lang.Exception` से आती है।

1. Definition (परिभाषा)

जब आपके कोड में कोई गड़बड़ी होती है (जैसे गलत इनपुट या फाइल का न मिलना), तो JVM उस विशिष्ट समस्या से संबंधित **Exception Class** का एक **Object** बनाता है। इस ऑब्जेक्ट में एरर का प्रकार और प्रोग्राम की स्थिति की जानकारी होती है।

Types of Java Exceptions (एक्सेप्शन के प्रकार)

जावा में एक्सेप्शन्स को मुख्य रूप से दो श्रेणियों में बांटा गया है, जो इस बात पर निर्भर करती हैं कि कंपाइलर उन्हें कब चेक करता है:

A. Checked Exceptions (कंपाइल-टाइम)

ये वे एक्सेप्शन्स हैं जिन्हें जावा कंपाइलर **Compile-time** पर ही चेक करता है। यदि आप इन्हें try-catch या throws से हैंडल नहीं करते, तो आपका प्रोग्राम कंपाइल ही नहीं होगा।

- Use Case:** ये आमतौर पर बाहरी कारकों (External factors) जैसे फाइल या नेटवर्क से संबंधित होते हैं।

B. Unchecked Exceptions (रनटाइम)

ये वे एक्सेप्शन्स हैं जो **Runtime** पर आते हैं। कंपाइलर इन्हें चेक नहीं करता, इसलिए इन्हें 'Runtime Exceptions' भी कहा जाता है।

- Use Case:** ये आमतौर पर प्रोग्रामिंग लॉजिक की गलतियों के कारण होते हैं।

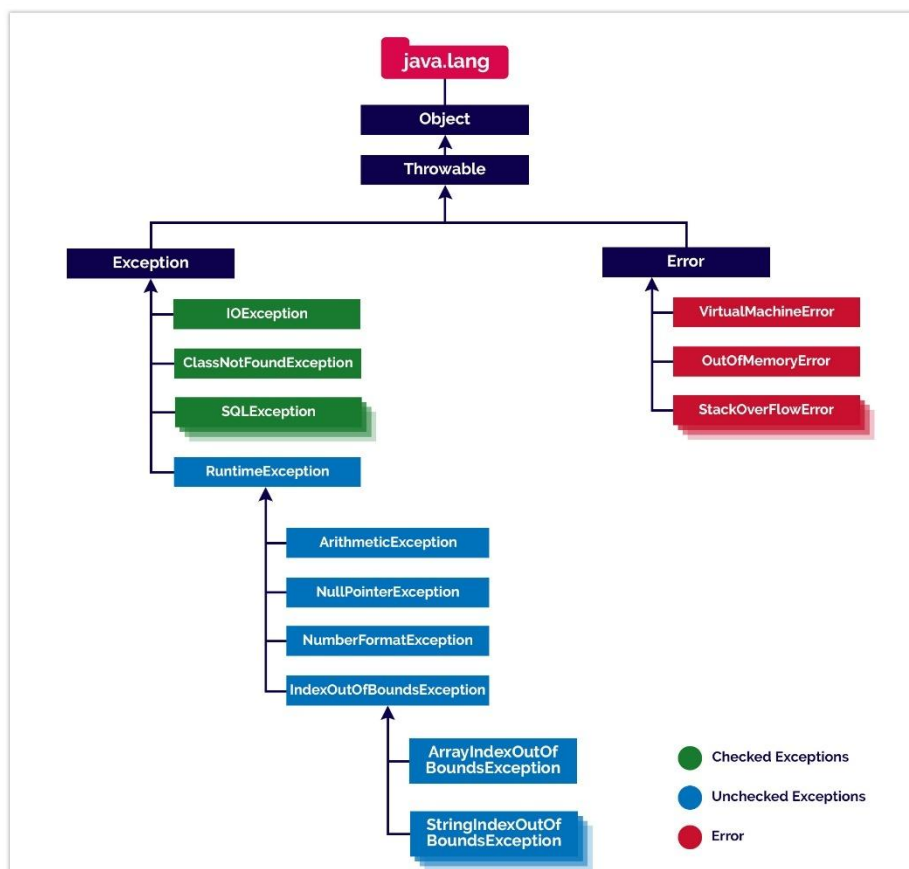
4. Common Exception Classes (प्रमुख उदाहरण)

Exception Type	Description (विवरण)	Category
ArithmeticException	जब गणितीय रूप से कुछ गलत हो (जैसे 10/0)।	Unchecked
NullPointerException	जब आप किसी null ऑब्जेक्ट के मेथड को कॉल करने की कोशिश करें।	Unchecked
ArrayIndexOutOfBoundsException	जब आप ऐरे की लिमिट (Index) से बाहर डेटा एक्सेस करें।	Unchecked
IOException	जब इनपुट-आउटपुट ऑपरेशन (जैसे फाइल पढ़ना) विफल हो जाए।	Checked
FileNotFoundException	जब आप ऐसी फाइल खोलना चाहें जो सिस्टम में मौजूद ही नहीं है।	Checked

Difference Between Java Error and Exception

जावा में Error और Exception दोनों ही Throwable क्लास की संतानें (Sub-classes) हैं, लेकिन इनके काम करने का तरीका और उद्देश्य पूरी तरह अलग है।

आधार (Basis)	Java Error (जावा एरर)	Java Exception (जावा एक्सेप्शन)
1. क्लास पाथ	यह java.lang.Error क्लास का हिस्सा है।	यह java.lang.Exception क्लास का हिस्सा है।
2. मूल कारण (Source)	यह बाहरी वातावरण या JVM की समस्याओं के कारण होता है।	यह प्रोग्रामिंग लॉजिक या गलत यूजर डेटा के कारण होता है।
3. सुधार (Recovery)	यह Irrecoverable (लाइलाज) है। प्रोग्राम को बंद करना ही पड़ता है।	यह Recoverable है। इसे संभालकर प्रोग्राम को जारी रखा जा सकता है।
4. हैंडलिंग (Handling)	इन्हें try-catch ब्लॉक से हैंडल करने की कोशिश नहीं करनी चाहिए।	इन्हें try-catch और finally का उपयोग करके आसानी से हैंडल किया जा सकता है।
5. प्रकार (Types)	ये हमेशा Unchecked होते हैं (रनटाइम पर ही पता चलते हैं)।	ये Checked (कंपाइल-टाइम) और Unchecked दोनों हो सकते हैं।
6. जिम्मेदारी	इसके लिए सिस्टम एडमिन या हार्डवेयर जिम्मेदार होता है।	इसके लिए प्रोग्रामर (यानी आप) जिम्मेदार होते हैं।
7. मुख्य उदाहरण	StackOverflowError, OutOfMemoryError	NullPointerException, ArithmeticException, IOException



Exception Handling (एक्सेप्शन हैंडलिंग)

Definition: यह जावा का एक ऐसा मैकेनिज्म (Mechanism) है जो रनटाइम एरर्स (Runtime Errors) को संभालता है। इसका मुख्य उद्देश्य प्रोग्राम को अचानक बंद (Crash) होने से बचाना और कोड के सामान्य प्रवाह (Normal Flow) को बनाए रखना है।

जब कोई गड़बड़ी होती है, तो JVM एक **Exception Object** बनाता है। हैंडलिंग का मतलब है उस ऑब्जेक्ट को पकड़ना और उसका समाधान करना।

2. Keywords and Their Uses (कीवर्ड्स और उनके उपयोग)

जावा में एक्सेप्शन हैंडलिंग के लिए **5 मुख्य कीवर्ड्स** का उपयोग किया जाता है:

Keyword	Use (उपयोग)
try	इसका उपयोग उस कोड ब्लॉक को घेरने के लिए किया जाता है जिसमें एक्सेप्शन आने की संभावना हो। इसे अकेले इस्तेमाल नहीं कर सकते।
catch	इसका उपयोग try ब्लॉक में आए एक्सेप्शन को पकड़ने और उसे हैंडल करने के लिए किया जाता है। यह try के ठीक बाद आता है।
finally	इसका उपयोग महत्वपूर्ण कोड (जैसे क्लोजिंग फाइल) को चलाने के लिए किया जाता है। यह चाहे एक्सेप्शन आए या न आए, हमेशा चलता है।
throw	इसका उपयोग किसी एक्सेप्शन को मैन्युअली (Explicitly) फेंकने के लिए किया जाता है।
throws	इसका उपयोग मेथड सिग्नेचर में किया जाता है यह बताने के लिए कि यह मेथड भविष्य में कौन-सा एक्सेप्शन दे सकता है।

3. Syntax (लिखने का तरीका)

आपके नोट्स के लिए यह स्टैंडर्ड सिंटैक्स है:

```
try {  
    // 1. Risky code (जहाँ गलती हो सकती है)  
}  
catch (ExceptionClassName e) {  
    // 2. Handling code (गलती होने पर क्या करना है)  
}  
finally {  
    // 3. Cleanup code (जो हमेशा चलेगा)  
}
```

4. Full Example (एक साथ सभी का उपयोग)

यहाँ एक प्रैक्टिकल उदाहरण है जो try, catch, और finally को एक साथ समझाता है:

Java

```
public class ExceptionDemo {
    public static void main(String[] args) {
        System.out.println("Program Started...");

        try {
            int a = 10;
            int b = 0;
            int result = a / b; // यहाँ ArithmeticException आएगा
            System.out.println("Result: " + result);
        }
        catch (ArithmeticException e) {
            // JVM इस ब्लॉक में आता है क्योंकि b = 0 है
            System.out.println("Error: Zero division is not allowed in Math.");
        }
        finally {
            // यह हमेशा प्रिंट होगा
            System.out.println("Cleaning up resources... Execution completed.");
        }

        System.out.println("Program finished safely.");
    }
}
```

Finally (हमेशा चलने वाला ब्लॉक)

उपयोग: इसका उपयोग क्लीनअप (Cleanup) के लिए किया जाता है। चाहे try में एरर आए या न आए, finally के अंदर का कोड हमेशा चलेगा।

Simple Example:

Java

```
public class FinallyDemo {
    public static void main(String[] args) {
        try {
            int a = 10 / 2; // कोई एरर नहीं है
            System.out.println("Result: " + a);
        } catch (Exception e) {
            System.out.println("Error caught.");
        } finally {
            System.out.println("Finally: I will always run!");
        }
    }
}
```

2. Throw

उपयोग: इसका उपयोग किसी विशिष्ट एक्सेप्शन को **मैन्युअली (Explicitly)** फेंकने के लिए किया जाता है। यह अक्सर if-else के साथ लॉजिक चेक करने के लिए इस्तेमाल होता है।

Simple Example:

Java

```
public class ThrowDemo {
    static void checkAge(int age) {
        if (age < 18) {
            // खुद से एक्सेप्शन पैदा करना
            throw new ArithmeticException("Aakash, you are not eligible to vote.");
        } else {
            System.out.println("Access granted!");
        }
    }

    public static void main(String[] args) {
        checkAge(15); // यह लाइन एरर थ्रो करेगी
    }
}
```

3. Throws (चेतावनी देना)

उपयोग: इसका उपयोग मेथड के सिग्नेचर में किया जाता है। यह कंपाइलर को बताता है कि "इस मेथड के अंदर कोई एक्सेप्शन आ सकता है, इसे इस्तेमाल करने वाला सावधान रहे"।

Simple Example:

```
public class ThrowsDemo {

    static void riskyMethod() throws InterruptedException {
        Thread.sleep(1000); // 1 सेकंड का इंतज़ार
        System.out.println("Waited for 1 second.");
    }

    public static void main(String[] args) {
        try {
            riskyMethod();
        } catch (InterruptedException e) {
            System.out.println("Interrupted!");
        }
    }
}
```

Quick Comparison Table

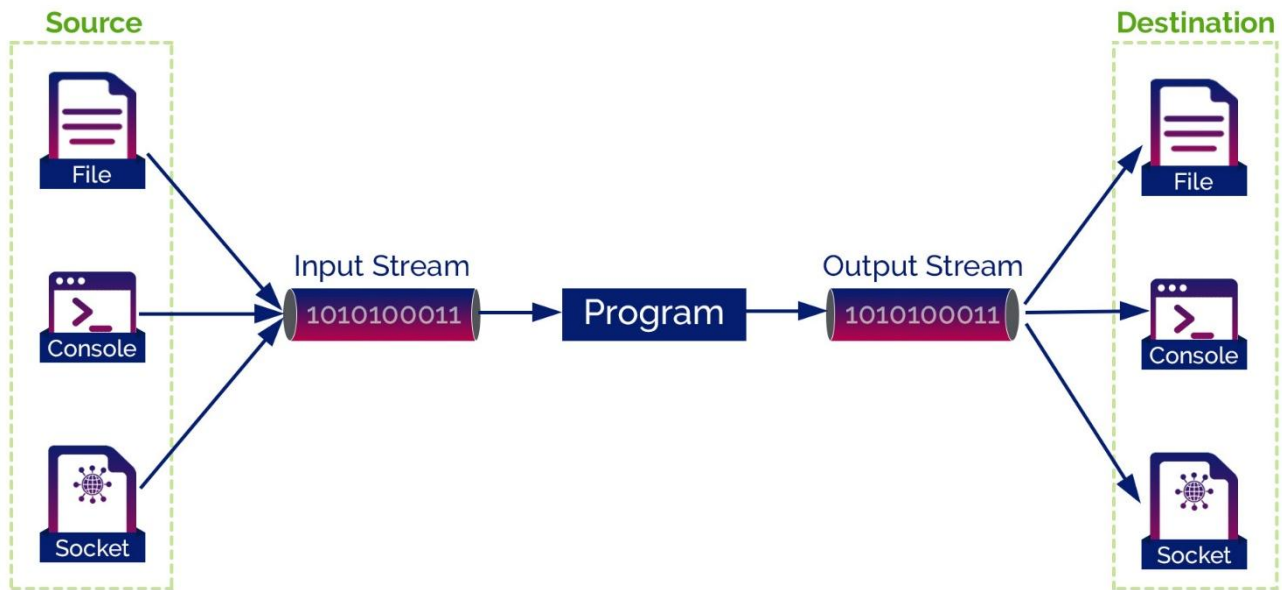
Keyword	Type	Purpose (उद्देश्य)
finally	Block	रिसोर्स क्लोज करना (जैसे फाइल या डेटाबेस)।
throw	Statement	एक नया एक्सेप्शन ऑब्जेक्ट "फेंकना"।
throws	Signature	आने वाले एक्सेप्शन की घोषणा (Declare) करना।

-
- **Throw** = "बम फेंकना" (Action)
 - **Throws** = "बम की चेतावनी देना" (Warning)
 - **Finally** = "धमाके के बाद सफाई करना" (Cleanup)

Stream in java

Java में, IO ऑपरेशन्स (operations) स्ट्रीम्स के कॉन्सेप्ट का उपयोग करके किए जाते हैं। आमतौर पर, स्ट्रीम का अर्थ डेटा का निरंतर प्रवाह (continuous flow of data) होता है। Java में, एक स्ट्रीम डेटा का एक लॉजिकल कंटेनर है जो हमें इससे डेटा पढ़ने और इसमें लिखने की अनुमति देता है। Java IO सिस्टम द्वारा एक स्ट्रीम को डेटा सोर्स (source) या डेटा डेस्टिनेशन (destination) जैसे कि कंसोल, फ़ाइल या नेटवर्क कनेक्शन से जोड़ा जा सकता है। स्ट्रीम-आधारित IO ऑपरेशन्स सामान्य IO ऑपरेशन्स की तुलना में तेज़ होते हैं।

Stream को java.io पैकेज में परिभाषित किया गया है।



Java में, स्ट्रीम-आधारित IO ऑपरेशन्स दो अलग-अलग स्ट्रीम्स— **input stream** और **output stream** का उपयोग करके किए जाते हैं। इनपुट स्ट्रीम का उपयोग इनपुट ऑपरेशन्स के लिए किया जाता है, और आउटपुट स्ट्रीम का उपयोग आउटपुट ऑपरेशन्स के लिए किया जाता है। Java स्ट्रीम बाइट्स (bytes) से बनी होती है।

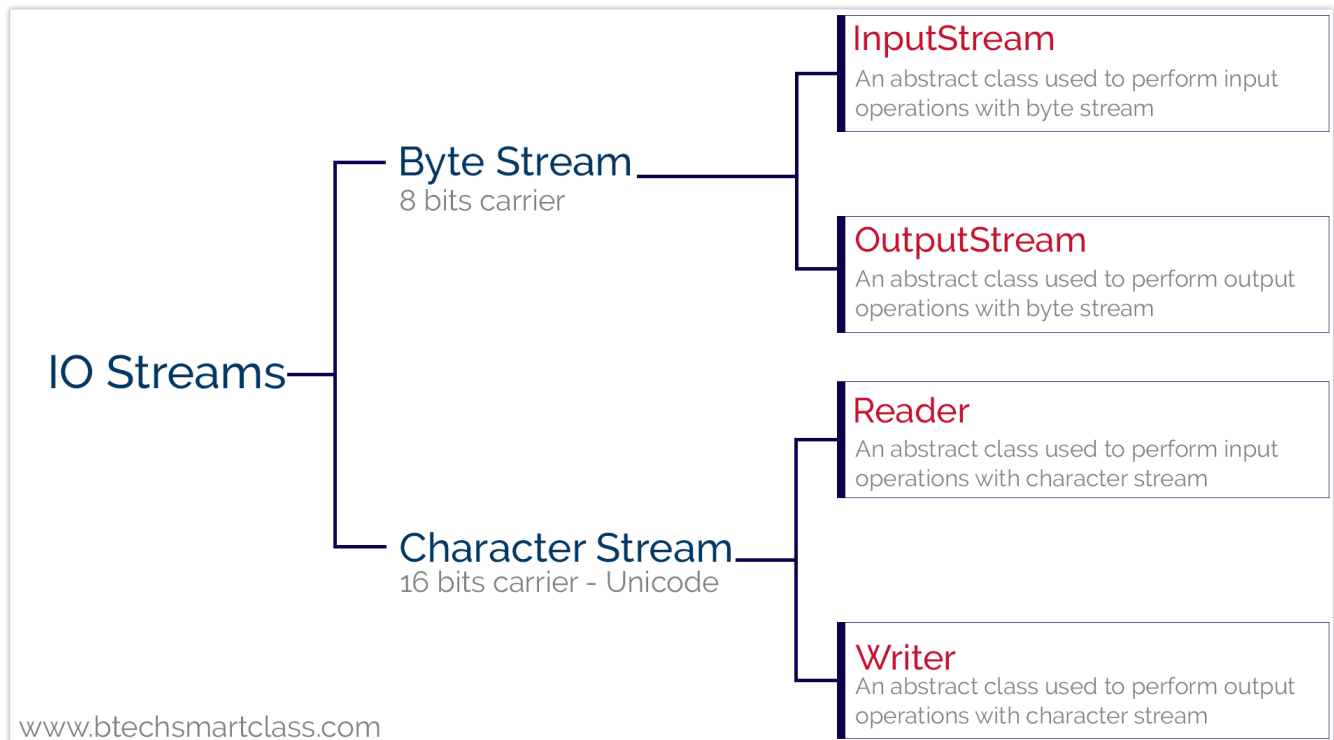
Java में, प्रत्येक प्रोग्राम अपने आप 3 स्ट्रीम्स बनाता है, और ये स्ट्रीम्स कंसोल से जुड़े होते हैं।

- **System.out:** कंसोल आउटपुट ऑपरेशन्स के लिए स्टैंडर्ड आउटपुट स्ट्रीम।
- **System.in:** कंसोल इनपुट ऑपरेशन्स के लिए स्टैंडर्ड इनपुट स्ट्रीम।
- **System.err:** कंसोल एरर आउटपुट ऑपरेशन्स के लिए स्टैंडर्ड एरर स्ट्रीम।

Java स्ट्रीम्स कई अलग-अलग प्रकार के डेटा का समर्थन करती हैं, जिनमें सरल बाइट्स, प्रिमिटिव डेटा टाइप, लोकलाइज्ड कैरेक्टर्स और ऑब्जेक्ट्स शामिल हैं।

Java दो प्रकार के स्ट्रीम्स प्रदान करता है, जो निम्नलिखित हैं:

- **Byte Stream**
- **Character Stream**



Byte Stream in java

Java में, Byte Stream का उपयोग 8-बिट बाइट्स के इनपुट और आउटपुट को हैंडल करने के लिए किया जाता है। यह बाइनरी डेटा को पढ़ने और लिखने के लिए सबसे उपयुक्त है, जैसे कि इमेज फाइल्स, ऑडियो, वीडियो या कोई भी ऐसी फाइल जिसमें रॉ डेटा (raw data) होता है।

Java में Byte Stream के लिए दो मुख्य एबस्ट्रैक्ट क्लासेज (abstract classes) का उपयोग किया जाता है:

1. **InputStream**: डेटा पढ़ने (Read) के लिए।
2. **OutputStream**: डेटा लिखने (Write) के लिए।

Example: FileInputStream & FileOutputStream

Byte Stream के सबसे सामान्य उदाहरण FileInputStream और FileOutputStream हैं, जिनका उपयोग फाइल ऑपरेशन्स के लिए किया जाता है।

```
import java.io.*;
```

```
public class ByteStreamExample {
    public static void main(String args[]) throws IOException {
        FileInputStream in = null;
        FileOutputStream out = null;
```

```
try {
    in = new FileInputStream("input.txt");
    out = new FileOutputStream("output.txt");
    int c;

    // एक-एक बाइट पढ़ना और लिखना
    while ((c = in.read()) != -1) {
        out.write(c);
    }
} finally {
    if (in != null) in.close();
    if (out != null) out.close();
}
}
```

Java में कई तरह की बाइट स्ट्रीम क्लासेज उपलब्ध हैं जैसे `BufferedInputStream`, `DataInputStream`, आदि, जो विभिन्न आवश्यकताओं को पूरा करती हैं।

Character Stream in java

Java में, Character Stream का उपयोग 16-बिट यूनिकोड (Unicode) कैरेक्टर्स के इनपुट और आउटपुट को हैंडल करने के लिए किया जाता है। चूंकि Java यूनिकोड का उपयोग करता है, इसलिए टेक्स्ट फाइलों को पढ़ने और लिखने के लिए Character Stream सबसे अच्छा तरीका है। यह अंतरराष्ट्रीय भाषाओं (जैसे हिंदी, चीनी, आदि) के कैरेक्टर्स को भी कुशलतापूर्वक संभाल सकता है।

Java में Character Stream के लिए दो मुख्य एबस्ट्रैक्ट क्लासेज (abstract classes) का उपयोग किया जाता है:

1. **Reader:** डेटा पढ़ने (Read) के लिए।
2. **Writer:** डेटा लिखने (Write) के लिए।

Reader Class

Reader क्लास का उपयोग कैरेक्टर्स के स्ट्रीम को पढ़ने के लिए किया जाता है। इसके कुछ महत्वपूर्ण मेथड्स निम्नलिखित हैं:

- **read():** यह स्ट्रीम से एक सिंगल कैरेक्टर पढ़ता है।
- **ready():** यह चेक करता है कि क्या स्ट्रीम पढ़ने के लिए तैयार है।
- **close():** यह रीडर स्ट्रीम को बंद कर देता है और उससे जुड़े रिसोर्स को फ्री कर देता है।

Writer Class

Writer क्लास का उपयोग कैरेक्टर्स के स्ट्रीम को लिखने के लिए किया जाता है। इसके कुछ महत्वपूर्ण मेथड्स निम्नलिखित हैं:

- **write(int c):** यह स्ट्रीम में एक सिंगल कैरेक्टर लिखता है।
- **write(String str):** यह स्ट्रीम में पूरी स्ट्रिंग लिखता है।
- **close():** यह राइटर स्ट्रीम को बंद कर देता है।

Example: FileReader & FileWriter

Character Stream के सबसे सामान्य उदाहरण `FileReader` और `FileWriter` हैं।

```
import java.io.*;
```

```
public class CharacterStreamExample {
    public static void main(String args[]) throws IOException {
        FileReader reader = null;
        FileWriter writer = null;

        try {
            reader = new FileReader("source.txt");
            writer = new FileWriter("destination.txt");
            int c;

            // एक-एक कैरेक्टर पढ़ना और लिखना
            while ((c = reader.read()) != -1) {
                writer.write(c);
            }
        } finally {
            if (reader != null) reader.close();
            if (writer != null) writer.close();
        }
    }
}
```

Byte Stream और Character Stream के बीच मुख्य अंतर

विशेषता	Byte Stream	Character Stream
यूनिट	8-बिट बाइट्स	16-बिट कैरेक्टर्स
उपयोग	बाइनरी डेटा (इमेज, वीडियो) के लिए	टेक्स्ट डेटा (टेक्स्ट फाइल्स) के लिए
मुख्य क्लासेज	InputStream, OutputStream	Reader, Writer

Why use BufferedReader instead of FileReader?

हालांकि FileReader का उपयोग सीधे फ़ाइल से डेटा पढ़ने के लिए किया जा सकता है, लेकिन बड़े डेटा के लिए यह कुशल (efficient) नहीं माना जाता है। इसीलिए प्रोफेशनल्स BufferedReader का उपयोग करने की सलाह देते हैं। इसके मुख्य कारण निम्नलिखित हैं:

1. Performance (प्रदर्शन)

FileReader सीधे डिस्क (Hard Disk) से एक-एक कैरेक्टर पढ़ता है। डिस्क से डेटा पढ़ना एक धीमी प्रक्रिया है। यदि आप 1000 कैरेक्टर पढ़ रहे हैं, तो FileReader डिस्क को 1000 बार एक्सेस करेगा। इसके विपरीत, BufferedReader एक **इंटरनल बफर (Internal Buffer)** का उपयोग करता है। यह डिस्क से एक साथ बहुत सारा डेटा (कैरेक्टर्स का ब्लॉक) पढ़कर बफर में रख लेता है। जब आप read() कॉल करते हैं, तो यह डिस्क के बजाय मेमोरी (RAM) से डेटा देता है, जो कि बहुत तेज़ होता है।

Feature	FileReader	BufferedReader
Speed	धीमा (Slow)	तेज़ (Fast)
Disk Access	हर कैरेक्टर के लिए डिस्क एक्सेस करता है।	एक बार में ब्लॉक पढ़ता है, डिस्क एक्सेस कम करता है।
Methods	read()	read(), readLine()
Efficiency	कम कुशल (Less Efficient)	अधिक कुशल (Highly Efficient)

Example of using BufferedReader

```
import java.io.*;

public class BufferedExample {
    public static void main(String args[]) throws IOException {
        // BufferedReader को FileReader के ऊपर रैप (wrap) किया जाता है
        FileReader fr = new FileReader("test.txt");
        BufferedReader br = new BufferedReader(fr);

        String line;
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }

        br.close();
        fr.close();
    }
}
```

सरल शब्दों में, FileReader एक साधारण मज़दूर की तरह है जो एक-एक ईंट उठाकर लाता है, जबकि BufferedReader एक ट्रॉली की तरह है जो एक साथ कई ईंटें ले आता है, जिससे समय और मेहनत दोनों बचती है।

Graphics Class in java

Java में, Graphics क्लास सभी ग्राफिक्स ऑपरेशन्स के लिए आधार (base) प्रदान करती है। यह क्लास java.awt पैकेज का हिस्सा है। Graphics क्लास का उपयोग स्क्रीन पर विभिन्न आकृतियों (shapes) जैसे रेखाएं (lines), आयत (rectangles), और वृत्त (circles) बनाने के लिए किया जाता है।

जब भी हमें कुछ ड्रा करना होता है, तो हम paint(Graphics g) मेथड का उपयोग करते हैं, जहाँ g Graphics क्लास का एक ऑब्जेक्ट है।

Java Graphics क्लास विभिन्न आकृतियों को ड्रा करने के लिए निम्नलिखित मेथड्स प्रदान करती है:

Drawing Lines

एक सीधी रेखा खींचने के लिए drawLine() मेथड का उपयोग किया जाता है। यह दो बिंदुओं \$(x1, y1)\$ और \$(x2, y2)\$ के बीच रेखा खींचता है।

- **Syntax:** void drawLine(int x1, int y1, int x2, int y2)

Drawing Rectangles

आयत (Rectangle) ड्रा करने के लिए Graphics क्लास दो मुख्य मेथड्स प्रदान करती है:

1. **drawRect():** यह केवल आयत की आउटलाइन (border) ड्रा करता है。
 - **Syntax:** void drawRect(int x, int y, int width, int height)
2. **fillRect():** यह आयत को वर्तमान रंग (current color) से भर देता है।
 - **Syntax:** void fillRect(int x, int y, int width, int height)

Drawing Circles & Ellipses

Java में सर्कल या एलिप्स (अंडाकार आकृति) ड्रा करने के लिए कोई अलग drawCircle() मेथड नहीं होता है। इसके बजाय, हम drawOval() मेथड का उपयोग करते हैं।

1. **drawOval():** यदि चौड़ाई (width) और ऊँचाई (height) अलग-अलग हों, तो यह एक एलिप्स बनाता है। यदि दोनों बराबर हों, तो यह एक सर्कल बनाता है।
 - **Syntax:** void drawOval(int x, int y, int width, int height)
2. **fillOval():** यह अंडाकार आकृति या सर्कल को रंग से भर देता है।
 - **Syntax:** void fillOval(int x, int y, int width, int height)

Graphics क्लास में ड्रॉइंग के लिए कोऑर्डिनेट सिस्टम (coordinate system) का उपयोग किया जाता है, जहाँ ऊपरी बाएँ कोने (top-left corner) को \$(0,0)\$ माना जाता है।

Example Program for Graphics Class

यह प्रोग्राम एक विंडो बनाता है जहाँ लाइन, रेक्टेंगल और सर्कल को ड्रा किया गया है:

```
import java.applet.*;
import java.awt.*;
public class GApplet extends Applet
{
    public void paint(Graphics g)
    {
        g.drawLine(20, 20, 500, 20);
        g.drawRect(20, 40, 200, 40);
        g.fillRect(300, 40, 200, 40);
        g.drawRoundRect(20, 100, 200, 40, 10, 10);
        g.fillRoundRect(300, 100, 200, 40, 10, 10);
        g.setColor(Color.RED);
        g.drawOval(20, 160, 200, 100);
        g.fillOval(300, 160, 200, 100);
    }
}
/*
<applet code="GApplet" height="500" width="700" border="1">
</applet>
*/
```

Output for the above program is as follows:

